

实验四：语义分析与类型系统

一、实验概述

本实验是编译原理实验系列的第四个实验。在实验三中，你的语法分析器已经能够构建抽象语法树（AST）。本实验的目标是：在 AST 的基础上进行语义分析——构建符号表、管理作用域、进行类型检查，检测并报告各类语义错误。

语法正确的程序未必有意义：变量可能未声明就使用，类型可能不匹配，函数调用参数个数可能错误。语义分析负责在编译期捕获这些错误，是保证程序正确性的重要一环，也是连接前端与后端的桥梁。

二、实验目标

- 理解符号表的构建与作用域管理（嵌套作用域、变量遮蔽）
- 掌握类型系统的设计与类型检查的实现
- 能够检测常见的语义错误并准确定位
- 掌握基于 AST 遍历的语义分析方法（属性文法 / 访问者模式）

三、实验内容

任务一：语义分析器实现（必做）

在实验三 AST 的基础上实现语义分析器。程序读入源文件（内部完成词法、语法分析后），进行语义检查，输出符号表信息与语义错误列表（JSON 格式）。

1. 命令行接口

```
./compiler check input.src
```

成功时输出（无语义错误，可附符号表信息）：

```
{  
  "errors": [],  
  "symbols": [  

```

```

    {"name": "x", "type": "int", "scope": "global", "line": 1},
    {"name": "f", "type": "int(int,int)", "scope": "global", "line": 3}
  ]
}

```

存在语义错误时输出：

```

{
  "errors": [
    {"type": "UNDECLARED", "symbol": "y", "line": 5, "col": 3},
    {"type": "TYPE_MISMATCH", "expected": "int", "found": "string",
      "line": 6, "col": 7},
    {"type": "ARG_COUNT_MISMATCH", "symbol": "f", "expected": 2,
      "found": 1, "line": 8, "col": 1}
  ]
}

```

2. 必检语义错误类型

错误类型	说明	示例
UNDECLARED	使用未声明的变量/函数	y = 1; （y 未声明）
REDECLARED	同一作用域内重复声明	int x; int x;
TYPE_MISMATCH	类型不兼容（赋值/运算/返回值）	int x = "abc";
ARG_COUNT_MISMATCH	函数调用参数个数错误	f(1) 但 f 需要 2 个参数
ARG_TYPE_MISMATCH	函数调用参数类型错误	f("s") 但形参为 int

以上为基本类型，你可根据自己语言的特性增加（如 NOT_A_FUNCTION、RETURN_TYPE_MISMATCH 等），并在报告中说明。

3. 必做功能

(1) 符号表与作用域管理

- 构建符号表，记录变量/函数的名称、类型、声明位置等信息
- 支持嵌套作用域（全局、函数、语句块），正确处理作用域的进入与退出
- 正确处理变量遮蔽（内层作用域同名变量覆盖外层）

(2) 声明检查

- 检测使用未声明的变量或函数（UNDECLARED）
- 检测同一作用域内的重复声明（REDECLARED）

(3) 类型检查

- 赋值类型兼容性检查
- 运算符操作数类型检查（如算术运算要求数值类型）
- 函数调用的参数个数与类型检查、返回值类型检查（如你的语言有函数）

(4) 错误定位与恢复

每个语义错误都需准确定位（行号、列号），并尽可能在一次分析中报告多个语义错误。

任务二：测试用例集（必做）

编写测试用例验证语义分析器，并在报告与演示中展示。要求 ≥ 10 组，覆盖以下场景：

编号	测试场景	最少用例数
T1	符号表与作用域（嵌套作用域、变量遮蔽）	2
T2	声明检查（未声明、重复声明）	2
T3	类型检查（赋值、运算、返回值）	2
T4	函数调用检查（参数个数、类型）	2
T5	正确程序（无语义错误）	2

任务三：必考场景验证（必做）

使用必考场景程序验证语义分析器（场景定义见《必考场景集（扩充版）》）：

- S1 - S5 的程序必须通过语义检查，无语义错误
- S8（词法、语法均合法，但含语义错误的场景）的语义错误必须被正确检测并报告

在演示与报告中展示每个场景程序的语义检查结果与符号表。

任务四：类型推导或复杂类型（选做，加分项）

实现以下任一扩展：(a) 局部变量类型推导（如根据初始化表达式推断变量类型）；
(b) 支持更复杂的类型（数组、结构体/记录、函数类型）及其类型检查。

四、交付物清单

序号	交付物	文件/格式	说明
1	语义分析器源代码	源代码文件	建议与前序实验共用代码库
2	构建/运行说明	README.md	如何编译和运行
3	类型系统设计文档	type-system.md 或写入报告	类型定义与兼容规则
4	测试用例集	tests/test_XX.src + .expected.json	≥ 10 组
5	场景 check 结果	scenarios/S1.check.json ~ S5.check.json + S8.check.json	正向 5 个 + 语义错误场景 S8
6	AI 协作对话记录	conversation/ 目录 (Markdown 或导出文件)	完整对话 + 每轮工作纪要
7	实验报告	report.pdf / report.docx	含完整功能演示流程（图文）

五、验收方式

本实验采用过程化验收，重点考察真实的开发过程以及功能实现的一致性，而非仅看最终代码。验收由助教完成，包含以下四个环节。四个环节相互印证：对话记录、实验报告、源代码三者必须能够对应起来。

环节一：实验演示检查

- 学生现场演示（或提交完整录屏），展示编译器本阶段功能的完整运行流程，覆盖本实验全部必做功能。
- 演示需同时体现：正常输入的正确处理，以及错误输入的检测与报告。
- 助教可在演示过程中临时更换输入用例、要求你解释某段代码的作用或某个设计决策的理由。

环节二：AI 协作过程记录检查

- 提交开发全过程中与智能体（Agent IDE / AI 助手，如 Claude Code、Cursor、CodeBuddy 等）的完整对话记录。
- 记录需包含每一轮交互的三部分：（1）你向智能体提出的问题或指令；（2）智能体对该轮对话的完整响应；（3）该轮的工作纪要——你做了什么决策、为什么这样做、如何验证结果。
- 工作纪要不要求包含代码输出本身，重点是设计思路、决策理由与验证过程。
- 课程鼓励使用 AI 工具，但你必须能够独立解释每一行代码与每一个设计选择。完整、连贯的对话记录是你真正参与开发的证据。

环节三：实验报告检查

- 实验报告必须包含完整的功能演示流程（图文并茂）：对每一项功能，给出运行截图加文字说明，完整展示从输入到输出的过程。
- 报告还需涵盖：设计决策、关键算法实现思路、遇到的问题及其解决方法。

环节四：代码审核

- 助教审核源代码，要求与环节二的对话记录、环节三的报告演示流程保持一致：
 - 代码中实现的功能，应能在报告的演示流程中找到对应的展示；

- 代码的实现方式与思路，应能由对话记录与工作纪要解释其来源。
- 若出现明显不一致（例如代码中存在报告未演示、对话记录也无法解释的功能），将影响本实验成绩。

六、实验报告要求

实验报告是验收的核心材料之一，必须包含以下内容：

（1）完整功能演示流程（图文并茂）

- 逐项展示本实验的每个功能：给出输入示例、运行命令、运行截图与输出结果，并配文字说明。
- 演示流程需与源代码、AI 对话记录一致（详见第五部分验收方式）。

（2）设计与实现说明

- 你的语言的类型系统是怎样的？有哪些类型？类型兼容规则是什么？
- 符号表与作用域是如何实现的？如何处理嵌套作用域与变量遮蔽？
- 类型检查是如何在 AST 上进行的（属性文法 / 访问者模式）？
- 语义错误的检测策略是什么？举例说明。
- 语义分析器如何与语法分析（实验三）、中间代码生成（实验五）对接。

（3）AI 协作过程小结

- 简述你如何与智能体协作完成本实验：哪些部分由你主导设计、哪些借助 AI 实现、你如何验证 AI 产出的正确性。

七、参考资料

- Aho, Lam, Sethi, Ullman. Compilers: Principles, Techniques, and Tools (Dragon Book), Chapter 5, 6
- Appel. Modern Compiler Implementation, Chapter 5
- Pierce. Types and Programming Languages, Chapter 1 - 9（类型系统进阶参考）